PEP 621 – Storing project metadata in pyproject.toml

Important

This PEP is a historical document. The up-to-date, canonical spec, pyproject.toml specification, is maintained on the PyPA specs page.

See the PyPA specification update process for how to propose changes.

Abstract

This PEP specifies how to write a project's core metadata in a pyproject.toml file for packaging-related tools to consume.

Motivation

The key motivators of this PEP are:

- Encourage users to specify core metadata statically for speed, ease of specification, unambiguity, and deterministic consumption by build back-ends
- Provide a tool-agnostic way of specifying metadata for ease of learning and transitioning between build back-ends
- Allow for more code sharing between build back-ends for the "boring parts" of a project's metadata

To speak specifically to the motivation for static metadata, that has been an overall goal of the packaging ecosystem for some time. As such, making it easy to specify metadata statically is important. This also means that raising the cost of specifying data as dynamic is acceptable as users should skew towards wanting to provide static metadata.

Requiring the distinction between static and dynamic metadata also helps with disambiguation for when metadata isn't specified. When any metadata may be dynamic, it means you never know if the absence of metadata is on purpose or because it is to be provided later. By requiring that dynamic metadata be specified, it disambiguates the intent when metadata goes unspecified.

This PEP does not attempt to standardize all possible metadata required by a build back-end, only the metadata covered by the core metadata specification which are very common across projects and would stand to benefit from being static and consistently specified. This means build back-ends are still free and able to innovate around patterns like how to specify the files to include in a wheel. There is also an included escape hatch for users and build back-ends to

use when they choose to partially opt-out of this PEP (compared to opting-out of this PEP entirely, which is also possible).

This PEP is also not trying to change the underlying core metadata in any way. Such considerations should be done in a separate PEP which may lead to changes or additions to what this PEP specifies.

Rationale

The design guidelines the authors of this PEP followed were:

- Define a representation of as much of the core metadata in pyproject.toml as is reasonable
- Define the metadata statically with an escape hatch for those who want to define it dynamically later via a build back-end
- Use familiar names where it makes sense, but be willing to use more modern terminology
- Try to be ergonomic within a TOML file instead of mirroring how build back-ends specify metadata at a low-level when it makes sense
- Learn from other build back-ends in the packaging ecosystem which have used TOML for their metadata
- Don’t try to standardize things which lack a pre-existing standard at a lower-level
- When metadata is specified using this PEP, it is considered canonical

Specification

When specifying project metadata, tools MUST adhere and honour the metadata as specified in this PEP. If metadata is improperly specified then tools MUST raise an error to notify the user about their mistake.

Data specified using this PEP is considered canonical. Tools CANNOT remove, add or change data that has been statically specified. Only when a field is marked as dynamic may a tool provide a “new” value.

Details

Table name

Tools MUST specify fields defined by this PEP in a table named [project]. No tools may add fields to this table which are not defined by this PEP or subsequent PEPs. For tools wishing to store their own settings in pyproject.toml, they may use the [tool] table as defined in PEP 518. The lack of a [project] table implicitly means the build back-end will dynamically provide all fields.

name

- Format: string
- Core metadata: Name (link)
    - Flit: module/dist-name (link)
    - Poetry: name (link)
    - Setuptools: name (link)

The name of the project.

Tools MUST require users to statically define this field.

Tools SHOULD normalize this name, as specified by PEP 503, as soon as it is read for internal consistency.

version

- Format: string
- Core metadata: Version (link)
    - Flit: N/A (read from a __version__ attribute) (link)
    - Poetry: version (link)
    - Setuptools: version (link)

The version of the project as supported by PEP 440.

Users SHOULD prefer to specify already-normalized versions.

description

- Format: string
- Core metadata: Summary (link)
    - Flit: N/A
    - Poetry: description (link)
    - Setuptools: description (link)

The summary description of the project.

readme

- Format: String or table
- Core metadata: Description (link)
    - Flit: description-file (link)
    - Poetry: readme (link)
    - Setuptools: long_description (link)

The full description of the project (i.e. the README).

The field accepts either a string or a table. If it is a string then it is the relative path to a text file containing the full description. Tools MUST assume the file's encoding is UTF-8. If the file path ends in a case-insensitive .md suffix, then tools MUST assume the content-type is text/markdown. If the file path ends in a case-insensitive .rst, then tools MUST assume the content-type is text/x-rst. If a tool recognizes more extensions than this PEP, they MAY infer the content-type for the user without specifying this field as dynamic. For all unrecognized suffixes when a content-type is not provided, tools MUST raise an error.

The readme field may also take a table. The file key has a string value representing a relative path to a file containing the full description. The text key has a string value which is the full description. These keys are mutually-exclusive, thus tools MUST raise an error if the metadata specifies both keys.

A table specified in the readme field also has a content-type field which takes a string specifying the content-type of the full description. A tool MUST raise an error if the metadata does not specify this field in the table. If the metadata does not specify the charset parameter, then it is assumed to be UTF-8. Tools MAY support other encodings if they choose to. Tools MAY support alternative content-types which they can transform to a content-type as supported by the core metadata. Otherwise tools MUST raise an error for unsupported content-types.

requires-python

- Format: string
- Core metadata: Requires-Python (link)
    - Flit: requires-python (link)
    - Poetry: As a python dependency in the [tool.poetry.dependencies] table (link)
    - Setuptools: python_requires (link)

The Python version requirements of the project.

license

- Format: Table
- Core metadata: License (link)
    - Flit: license (link)
    - Poetry: license (link)
    - Setuptools: license, license_file, license_files (link)